

JavaScript: this, bind, and arrow functions

1) What is `this` in strict mode inside a regular function invoked directly (`fn()`) in the browser? And what about outside strict mode?

1. `this` בפונקציה רגילה שנקראת ישירות בדפדפן במצב `strict` הוא `undefined`. כאשר לא משתמשים ב- `strict` ה- `this` הוא `window`.

2) What is the core difference between `this` in a regular function versus `this` in an arrow function?

2. ההבדל העיקרי בין `this` בפונקציה רגילה לבין `this` בפונקציות `arrow` הוא שבפונקציה רגילה ההתייחסות של `this` משתנה בהתאם לשימוש של הפונקציה: אם הפונקציה שייכת לאובייקט ה- `window`, ה- `this` מתייחס ל- `window` (או שהוא `undefined` במצב `strict`), אם הפונקציה שייכת לאובייקט אחר, ה- `this` מתייחס לאותו אובייקט אחר, אם הפונקציה שייכת ל- `event` אז `this` מתייחס לאלמנט שקיבל את ה- `event`. במופע של `class` ה- `this` יתייחס למופע. לעומת זאת כש- `this` נמצא בפונקציית `arrow` הוא תמיד מתייחס לסביבה בה הוגדרה הפונקציה אם זה ה- `window` אז אליו אבל אם פונקציית ה- `arrow` הוגדרה בתוך פונקצייה רגילה, כמו למשל בתוך מטודה, היא תירש ממנה את ה- `this` שלה האובייקט אליו המטודה שייכת. גם אם פונקציית ה- `arrow` תיקרא במקום אחר כמו ה- `window`, ה- `this` עדיין יתייחס לאובייקט, אליו המטודה שבה היא נוצרה, שייכת.

3) What do `call`, `apply`, and `bind` do — and how do they differ from each other?

3. שלוש המטודות `call`, `applay` ו- `bind` מאפשרות לאובייקט להשתמש במטודה של אובייקט אחר ע"י כך שהן גורמות ל- `this` שבמטודה שבה משתמשים להתייחס לאובייקט המשתמש. שלושתן יכולות לקבל ארגומנטים אבל `apply` מקבלת אותם כמערך. שתי המטודות `call` ו- `apply` מופעלות מיד על האובייקט החדש. `bind` יוצרת פונקצייה חדשה שאפשר להשתמש בה גם מאוחר יותר.

4) Why doesn't `bind` affect an arrow function?

4. ל- `bind` אין השפעה על פונקציית `arrow` כי ה- `this` בפונקציית `arrow` תמיד עובר בתורשה מהסביבה בה פונקציית ה- `arrow` הוגדרה. ככתוב בתשובה לשאלה 3, `bind` משנה את האובייקט אליו מתייחס ה- `this`. זה אפשרי בפונקציות רגילות שבהן ההתייחסות ניתנת לשינוי (ככתוב בתשובה לשאלה 2) אבל בפונקציית `arrow` אין אפשרות לשינוי התייחסות ולכן `bind` לא משפיעה על פונקציות `arrow`.

5) Can an arrow function be used as a constructor with `new`? Why or why not?

5. פונקציית `arrow` לא יכולה לשמש כ- `constructor` עם `new` כי כל המטרה של פונקציות כאלה הן לבנות אובייקטים בדמות האובייקט המקורי בעל ה- `constructor` אבל שה- `this` שלהם יתייחס לעצמם. פונקציית `arrow`, ככתוב בתשובות הקודמות, לא מאפשרת שינוי התייחסות של `this` ולכן המטרה של פונקציית ה- `constructor`, שמופעלת ע"י `new`, לא תושג.

6) How is `this` determined when passing an object method as a callback?

6. כשמשתמשים במטודה כ- `callback` ה- `this` מאבד את ההקשר שהיה לו ומתייחס לאובייקט לפי איך שהפונקציה נקראה אם ה- `window` קרא אז ל- `window`. אם מטודה הפעילה את ה- `callback` אז ה- `this` יתייחס לאובייקט שהמטודה המפעילה שייכת לו. אם ה- `callback` הופעלה ע"י `event` אז `this` יתייחס לאלמנט שקיבל אותה.

JavaScript: this, bind, and arrow functions

7) Contrast `this` in a DOM event listener defined with a regular function vs. with an arrow function.

7. כמו שעניתי בתשובה 2, במקרה של event בפונקציה רגילה this מתייחס לאלמנט שקיבל את ה-event בפונקציית arrow ה-this יתייחס ל-this בסביבה בה פונקציית ה-arrow נוצרה.

8) How does `this` behave inside `setTimeout`/`setInterval` callbacks?

8. this בתוך setTimeout ו-setInterval מתייחס ל-window שהוא האובייקט אליו שייכות המטודות.

9) In a `class`, how does `this` differ between a regular method and a class field defined as an arrow function (e.g., `handle = () => {}`)?

9. ההבדל ב-this ב-class כשמגדירים מטודה רגילה או מטודה כפונקציית arrow בא לידי ביטוי רק כשמנסים להפעיל את המטודה באמצעות callback כמו setTimeout במקרה הזה אם המטודה הוגדרה כפונקציה רגילה ה-this יאבד את הקשרו למופע של ה-class והתוצאה תהיה undefined אם הפונקציה הוגדרה כ-arrow הקשרו של ה-this ישמר והמטודה תציג את התוצאה כשה-this מתייחס לאובייקט המופע של הקלאס.

10) What is `this` at the top level of an ES module (`type="module"`) vs. a classic script?

10. בסקריפט רגיל הרמה הכי עליונה היא window וה-this מתייחס אליו. לעומת זאת modules תמיד מבוצעים במצב strict ואז this מוגדר כ-undefined.

What prints and why

```
11)
"use strict";
function f() { console.log(this); }
f();
```

11. הפונקציה נקראה ב-window לכן ה-this מתייחס אליו. בגלל שנקבע שהסקריפט במצב strict שבו this שמתייחס ל-window מוגדר כ-undefined, הפונקציה שתפקידה להדפיס את ה-this ב-console תדפיס undefined.

```
12)
const obj = {
  x: 42,
  getX() { return this.x; }
};
const g = obj.getX;
console.log(g());
```

12. כשהגדרנו את המשתנה g למעשה יצרנו פונקציה חדשה בשם g שעושה את מה שהמטודה של obj עושה. אבל הפונקציה g לא נוצרה ב-obj היא נוצרה ב-window ומי שקורא לה הוא window ולכן ה-this שלה מתייחס ל-window ול-window אין הגדרה ל-x ולכן תוצאת ההדפסה היא undefined.

JavaScript: this, bind, and arrow functions

13)

```
const obj = {  
  x: 42,  
  getX() { return this.x; }  
};  
const g = obj.getX.bind(obj);  
console.log(g());
```

13. בהמשך לתשובה 12 הפעם יצרנו פונקציה חדשה אבל הפעלנו עליה את המטודה bind כפי שהסברתי בתשובה 3 המטודה bind גרמה ל- this בפונקציה g להתייחס לאובייקט obj ל- obj ה- x מוגדר כ-42 ולכן בניגוד למה שקרה בשאלה הקודמת יודפס 42.

14)

```
const obj = {  
  x: 1,  
  inc: () => { this.x++; }  
};  
obj.inc();  
console.log(obj.x);
```

14. כשמטודה של אובייקט היא פונקציה רגילה ה- this מתייחס לאובייקט אליו היא שייכת. אבל כשהמטודה היא פונקציית arrow היא יורשת את this מהסביבה בה היא נוצרה. האובייקט עצמו הוא לא סביבה ולכן הסביבה היא הסביבה בה האובייקט הוגדר ובמקרה שלנו ה- window כשאנחנו מפעילים את המטודה inc ה- x של window הוא זה שנוסף לו 1 וה- x של obj אותו אנחנו מדפיסים מיד אחר כך נשאר ללא שינוי כלומר מודפס 1.

15)

```
const obj = {  
  x: 1,  
  inc() {  
    setTimeout(function() {  
      this.x++;  
      console.log('in timeout', this.x);  
    }, 0);  
  }  
};  
obj.inc();
```

15. הפעלנו פונקציה רגילה בתוך מטודת setTimeout ה- this במקרה הזה, כמו שלמדנו בתשובה 8, מתייחס ל- window ה- x של window הוא undefined וכשמוסיפים מספר ל- undefined מקבלים NaN ולכן יודפס NaN in timeout.

JavaScript: this, bind, and arrow functions

```
16)
const obj = {
  x: 1,
  inc() {
    setTimeout(() => {
      this.x++;
      console.log('arrow timeout', this.x);
    }, 0);
  }
};
obj.inc();
```

16. בהמשך לתשובה הקודמת, הפעם בתוך המטודה `inc` הפעלו פונקציית `arrow` בתוך מטודת `setTimeout`. ה-`this` בתוך פונקציית ה-`arrow` הוא ירושה מהסביבה בה נוצרה הפונקציה (לא משנה מי קרא לה), הסביבה במקרה הזה היא המטודה `inc` שה-`this` שלה מתייחס לאובייקט אליו היא שייכת כלומר ל-`obj` לכן כשנפעיל את יתווסף 1 ל-`x` של `obj` יודפס `2 arrow timeout`.

```
17)
const obj = {
  val: 10,
  m: function () { return this.val; }
};
const h = obj.m.bind({ val: 99 });
console.log(h.call({ val: 7 }));
```

17. כשיצרנו את המשתנה `h`, יצרנו פונקצייה חדשה על בסיס המטודה `m` שה-`this` שלה הוא האובייקט `{val:99}` בגלל ה-`bind` אחר כך הדפסנו אותה כשהיא מופעלת ע"י המטודה `call` המטודה הזאת גרמה להפעלתה המיידית של `h` אבל שוב בגלל ה-`bind` ה-`this` לא השתנה כך שהערך שיוחזר ישאר 99 וזה מה שיודפס.

```
18)
const obj = {
  n: 5,
  m: () => this.n
};
console.log(obj.m());
```

18. בגלל שהמטודה `m` הוגדרה כפונקציית `arrow` ה-`this` אליו היא מתייחסת הוא ה-`this` של הסביבה בה היא נוצרה שהוא במקרה הזה `window` ה-`n` של `window` לא מוגדר, בהדפסה נפעיל את המטודה ונקבל `undefined`.

JavaScript: this, bind, and arrow functions

19) (DOM)

```
const button = document.createElement('button');
button.textContent = 'Click';
button.addEventListener('click', function () {
  console.log(this.tagName);
});
button.click();
```

19. הקוד הזה מייצר כפתור שרשום עליו Click מוסיפים לו Event listener כשלוחצים עליו תתבצע קולבק שתדפיס לקונסולה את שם התגית של this פונקציית הקולבק היא פונקצייה רגילה שארוע הלחיצה על הכפתור מפעיל אותה במקרה הזה ה- this הוא המטרה של הארוע כלומר הכפתור עצמו התגית שלו היא BUTTON ולכן זה מה שיודפס בקונסולה כשנלחץ על הכפתור.

20) (DOM)

```
const button = document.createElement('button');
button.addEventListener('click', () => {
  console.log(this === button);
});
button.click();
```

20. הקוד הזה מייצר כפתור. מוסיפים לו Event listener כשלוחצים עליו תתבצע קולבק שתדפיס לקונסולה את תוצאת הבדיקה (true או false) האם this הוא הכפתור עצמו. פונקציית הקולבק היא פונקציית arrow שהוגדרה בתוך הכפתור במקרה הזה ה- this הוא הסביבה בה הוגדרה הפונקצייה הכפתור עצמו לא נחשב סביבה הסביבה היא המקום בו הכפתור נמצא כלומר ה- window ולכן תוצאת הבדיקה תהיה false וזה מה שיודפס בקונסולה כשנלחץ על הכפתור.

JavaScript: this, bind, and arrow functions

Fix / improve the code (explain the issue and write a fix)

21) Class method "lost" as a callback:

```
class Counter {  
  constructor() { this.c = 0; }  
  inc() { this.c++; }  
}  
const counter = new Counter();  
setInterval(counter.inc, 100);
```

— Fix so the counter actually increments.

21. בקוד הנתון יש לנו קלאס Counter שלכל מופע שלו יהיה c ששווה ל-0. לקלאס יש גם מטודה שכל פעם שהיא נקראת היא מוסיפה 1 ל-c. בהמשך הקוד יוצרים מופע לקלאס וקוראים לו counter. ואז מפעילים את המטודה inc שלו כל עשירית שנייה ע"י שימוש ב-setInterval. המטודה setInterval היא מטודה של window והוא מי שקורא לה ולא counter. ל-window אין c לכן המטודה inc תוסיף בפעם הראשונה 1 ל-undefined i ונקבל NaN ובהמשך תוסיף כל עשירית שנייה 1 ל-NaN כך שנקבל שוב ושוב NaN

31 NaN

[doogmaot.js:216](#)

הפתרון הוא להגדיר את המטודה כפונקציית arrow במקום כפונקציית רגילה וכך ה- this יתייחס תמיד למופע של ה-class לא משנה מי יקרא למטודה של הקלאס והמונה יעבוד.

כדי שאפשר יהיה לראות את המונה הוספתי הוראה של console.log (שורה 216) בקוד המקורי אי אפשר לראות את התוצאה של הפעלת הפונקציה

```
214 class Counter {  
215   constructor() { this.c = 0; }  
216   inc = () =>{ console.log(this.c++); }  
217 }  
218 const counter = new Counter();  
219 setInterval(counter.inc, 100);
```

0	doogmaot.js:216
1	doogmaot.js:216
2	doogmaot.js:216
3	doogmaot.js:216

JavaScript: this, bind, and arrow functions

22) DOM listener using arrow but expecting element `this`:

```
button.addEventListener('click', () => {  
  this.disabled = true;  
});
```

— Fix so it works.

22. כמו שראינו בתשובה הקודמת ה-`this` שבפונקציית `arrow` שהוגדרה בתוך כפתור מתייחס לסביבה בה הכפתור נמצא. אם רוצים שה-`this` יתייחס לכפתור כדי שנוכל לנטרל אותו. צריך פשוט להגדיר את פונקציית הקולבק שבה נמצא ה-`this` כפונקצייה רגילה במקרה הזה ה-`this` יתייחס למטרת הארוע כלומר לכפתור עצמו וכך נוכל לנטרלו בלחיצה עליו.

```
213  const button = document.getElementById('button')  
214  button.addEventListener('click', function(){  
215    (() => this.disabled = true)()  
216  });
```

23) Sum with inner function that loses `this`:

```
const cart = {  
  items: [1,2,3],  
  sum() {  
    let total = 0;  
    this.items.forEach(function(n) {  
      total += n * this.multiplier; // this?  
    });  
    return total;  
  },  
  multiplier: 2  
};
```

— Fix in two different ways (e.g., `bind` and arrow).

JavaScript: this, bind, and arrow functions

23) Sum with inner function that loses `this`:

```
const cart = {  
  items: [1,2,3],  
  sum() {  
    let total = 0;  
    this.items.forEach(function(n) {  
      total += n * this.multiplier; // this?  
    });  
    return total;  
  },  
  multiplier: 2  
};
```

— Fix in two different ways (e.g., `bind` and arrow).

23. א. תיקון עם bind

```
221  const cart = {  
222    items: [1,2,3],  
223    sum() {  
224      let total = 0;  
225      this.items.forEach(function(n) {  
226        total += n * this.multiplier;  
227      }.bind(cart));  
228      return total;  
229    },  
230    multiplier: 2  
231  };  
232  console.log(cart.sum());
```

12

[doogmaot.js:232](#)

23. ב. תיקון עם פונקציית arrow

```
234  const cart = {  
235    items: [1,2,3],  
236    sum() {  
237      let total = 0;  
238      this.items.forEach((n) => {  
239        total += n * this.multiplier;  
240      });  
241      return total;  
242    },  
243    multiplier: 2  
244  };  
245  console.log(cart.sum());
```

12

[doogmaot.js:245](#)

JavaScript: this, bind, and arrow functions

24) Misusing `bind` on an arrow:

```
const obj = { x: 3 };  
const fn = () => this.x;  
const bound = fn.bind(obj);  
console.log(bound());
```

— Explain and correct to demonstrate proper binding.

24. בפונקציית arrow ה-`this` מתייחס תמיד לסביבה בה הפונקציה הוגדרה לא משנה מי קרא לפונקציה. `bind` מאלצת את הפונקציה להתייחס ל-`this` כאילו האובייקט שנשר על ידה קרא לה. `bind` חסרת השפעה על פונקציית arrow. אם רוצים ש-`bind` תעבוד צריך להפעיל אותה על פונקציית רגילה. התיקון הוא להפוך את פונקציית ה-`arrow` לפונקציה רגילה.

```
250  const obj = { x: 3 };  
251  function fn(){return this.x}  
252  const bound = fn.bind(obj);  
253  console.log(bound());
```

JavaScript: this, bind, and arrow functions

25) React/TS (conceptual): Why do we often prefer a class field arrow (`handleClick = () => {}`) over a regular method + `this.handleClick = this.handleClick.bind(this)` in the constructor? Provide a short code example.

25. בקוד הדוגמא שכתבתי יצרתי קלאס Car שיש לו מטודה statement שכותבת משפט שמבוסס על this. הגדרתי אותה כפונקציית arrow ולכן גם אם היא נקראת ע"י ה-window (בקוד היא נקראת ע"י ה-window באמצעות setTimeout) (שורה 267) עדיין נקבל את התוצאה שציפינו לה כאילו היא נקראה ע"י המופע myCar

```
258 class Car {
259   constructor(name, color) {
260     this.name = name
261     this.color = color
262   }
263   statement = () => {console.log(`I have a ${this.color} ${this.name}`)};
264 }
265
266 const myCar = new Car('Tesla','white');
267 setTimeout(myCar.statement,3000)
```

I have a white Tesla

[doogmaot.js:263](#)

לו הגדרתי אותה כפונקצייה רגילה ה- this היה מתייחס ל- window והיינו מקבלים תוצאה לא רצויה:

```
268 class Car {
269   constructor(name, color) {
270     this.name = name
271     this.color = color
272     // this.statement =this.statement.bind(this)
273   }
274   statement() {console.log(`I have a ${this.color} ${this.name}`)}
275 }
276
277 const myCar = new Car('Tesla','white');
278 setTimeout(myCar.statement,3000)
```

I have a undefined

[doogmaot.js:274](#)

JavaScript: this, bind, and arrow functions

כדי לפתור את זה נהיה חייבים לקשור את המטודה בתוך הקונסטרקטור למופע של הקלאס (שורה 272)

```
268 class Car {  
269   constructor(name, color) {  
270     this.name = name  
271     this.color = color  
272     this.statement = this.statement.bind(this)  
273   }  
274   statement() {console.log(`I have a ${this.color} ${this.name}`)}  
275 }  
276  
277 const myCar = new Car('Tesla','white');  
278 setTimeout(myCar.statement,3000)
```

I have a white Tesla

[doogmaot.js:274](#)

לו היו לנו מטודות נוספות לכל אחת מהן היינו צריכים להוסיף שורה בקונסטרקטור שתקשור אותן למופע של הקלאס דבר שמסרביל את הקוד של מטודת הקונסטרקטור, שמופעלת כל פעם שנוצר מופע חדש, דבר לא רצוי בכלל. לכן נעדיף להשתמש בפונקציית arrow שתאפשר השארת הקונסטרקטור נקי מסירבולים מיותרים.

JavaScript: this, bind, and arrow functions

26) Implement `once(fn, ctx?)` which returns a version that can run only once and invokes `fn` with `this = ctx` if provided, otherwise preserves the caller's `this`:

```
const log = function(a,b){ console.log(this.prefix, a+b); };
const onceLog = once(log, { prefix: '[sum]' });
onceLog(1,2); // prints once
onceLog(3,4); // does not print
```

```
283 function once(fn, ctx) {
284     let counter = 0
285     return function(x,y) {
286         if(counter == 1) return;
287         counter++
288         ctx ? fn.call(ctx, x ,y) : fn.call(this, x ,y)
289     }
290 }
291
292 const log = function(a,b){ console.log(this.prefix, a+b); };
293 const onceLog = once(log, { prefix: '[sum]' });
294 onceLog(1,2); // prints once
295 onceLog(3,4); // does not print
```

[sum] 3

[doogmaot.js:292](#)

27) Implement `mapWith(arr, methodName)` which returns the result of calling that method on each item, where the item becomes the method's `this`:

```
const arr = ['a','bb','ccc'];
mapWith(arr, 'toUpperCase'); // ['A','BB','CCC']
```

```
301 const arr = ['a','bb','ccc']
302
303 function mapWith(arr, methodName) {
304     console.log(arr.map(text=>text[methodName]()));
305 }
306
307 mapWith(arr, 'toUpperCase'); // ['A','BB','CCC']
```

▶ (3) ['A', 'BB', 'CCC']

[doogmaot.js:304](#)

JavaScript: this, bind, and arrow functions

28) Implement `bindAll(obj, .methodNames)` that binds the listed methods to `obj` (handy before passing callbacks).

28. לצורך ההדגמה הפעלתי את המטודות 3 פעמים. פעם ראשונה כרגיל. פעם שניה כשהן נקראות ע"י `setTimeout` ובכך ה-`this` אובד ומקבלים תוצאות לא רצויות. ופעם שלישית באמצעות `setTimeout` אבל אחרי שהפעלתי עליהן את הפונקצייה `bindAll` שכתבתי.

```
320 const orgObj = {
321   firstName: 'Josef',
322   lastName: 'Lion',
323   fullName(){console.log(this.firstName + " " + this.lastName)},
324   greetHello(){console.log(`Hello ${this.firstName}!`)},
325   greetBye(){console.log(`Goodbye ${this.firstName}!`)}
326 }
327 orgObj.fullName()
328 orgObj.greetHello()
329 orgObj.greetBye()
330
331
332 function bindAll(obj, arrOfMethods) {
333   arrOfMethods.forEach(methodName => {
334     obj[methodName] = obj[methodName].bind(obj)
335   });
336 }
337
338 setTimeout(orgObj.fullName, 3000)
339 setTimeout(orgObj.greetHello, 3000)
340 setTimeout(orgObj.greetBye, 3000)
341
342 bindAll(orgObj, ['greetBye','greetHello','fullName'])
343
344 setTimeout(orgObj.fullName, 3000)
345 setTimeout(orgObj.greetHello, 3000)
346 setTimeout(orgObj.greetBye, 3000)
```

Josef Lion	doogmaot.js:323
Hello Josef!	doogmaot.js:324
Goodbye Josef!	doogmaot.js:325
undefined undefined	doogmaot.js:323
Hello undefined!	doogmaot.js:324
Goodbye undefined!	doogmaot.js:325
Josef Lion	doogmaot.js:323
Hello Josef!	doogmaot.js:324
Goodbye Josef!	doogmaot.js:325